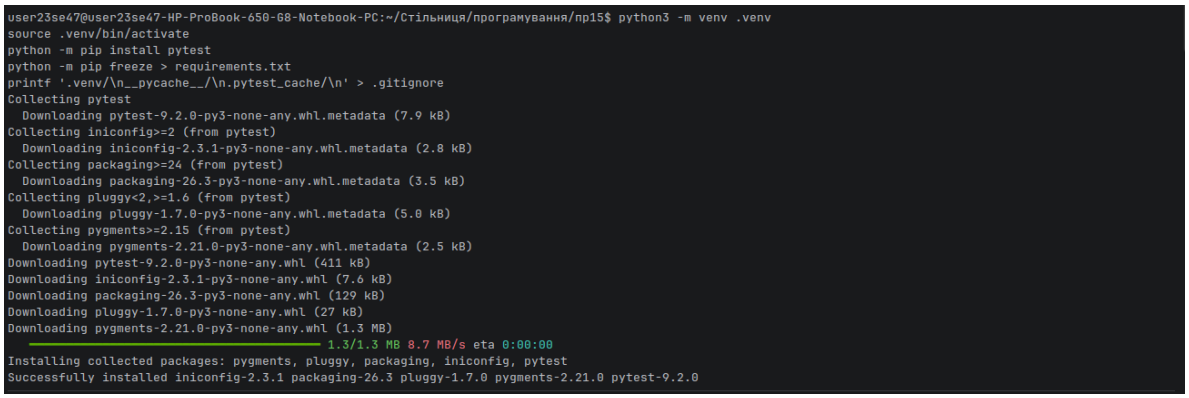
Захарчук Софія ІТ-32, програмування практична робота №15

Тема: Тестування коду: assert, pytest, параметризація та фікстури

Мета роботи: навчитися писати unit-тести до власних функцій за допомогою assert і pytest, перевіряти генерацію винятків, параметризувати тести та використовувати фікстуру tmp_path для роботи з файлами.

Підготовка

Я створила каталог пр15 з віртуальним середовищем .venv, встановила pytest, зберегла залежності у requirements.txt, створила .gitignore і скопіювала в каталог data/expenses.csv з роботи 22.



Завдання 1. Модуль з функціями

У файлі expenses.py я написала три функції, які нічого не друкують, а повертають результат. Функція parse_line розбирає рядок виду 2026-09-21;food;85.50;lunch у словник з полями date, category, amount і comment; для неправильної кількості полів або нечислової суми вона генерує ValueError. Функція total_by_category повертає словник {category: total} із сумами, округленими до двох знаків. Функція load_expenses читає файл, пропускає коментарі та порожні рядки і повертає список словників. Команда python -c "import expenses" нічого не виводить.

```
from pathlib import Path


def parse_line(line: str) -> dict:
    """Parse 'date;category;amount;comment' into a dict. Raise ValueError if bad."""
    parts = line.strip().split(";")
    if len(parts) != 4:
        raise ValueError(f"expected 4 fields, got {len(parts)}: {line!r}")
    date, category, amount, comment = (p.strip() for p in parts)
    try:
        amount = float(amount.replace(",", "."))
    except ValueError:
        raise ValueError(f"amount is not a number: {amount!r}") from None
    return {"date": date, "category": category, "amount": amount, "comment": comment}


def total_by_category(records: list[dict]) -> dict[str, float]:
    """Return {category: total}, totals rounded to 2 decimals."""
    totals: dict[str, float] = {}
    for record in records:
        category = record["category"]
        totals[category] = totals.get(category, 0.0) + record["amount"]
```

```
    return {category: round(total, 2) for category, total in totals.items()}


def load_expenses(path) -> list[dict]:
    """Read a file, skip comments and empty lines, return a list of records."""
    records = []
    for line in Path(path).read_text(encoding="utf-8").splitlines():
        line = line.strip()
        if not line or line.startswith("#"):
            continue
        records.append(parse_line(line))
    return records
```

Завдання 2. Перші тести з assert

У файлі test_expenses.py я написала три тести для parse_line: перевірка категорії та суми, перевірка всіх полів словника і перевірка того, що сума має тип float. Команда pytest -v показала, що всі 13 тестів пройдено (разом з тестами з наступних завдань). Потім я навмисно зламала функцію: замінила "amount": amount на "amount": str(amount), тобто функція почала повертати суму як рядок. Тести це виявили: 10 тестів впало з AssertionError, наприклад assert '85.5' == 85.5. Після цього я повернула файл із резервної копії.

```
from pathlib import Path

import pytest

from expenses import load_expenses, parse_line, total_by_category

# 5 рядків з мого data/expenses.csv
CASES = [
    ('2026-09-21;coffee;45.00;coffee before lecture', 'coffee', 45.0),
    ('2026-09-21;food;38.00;bread and cheese', 'food', 38.0),
    ('2026-09-21;transport;16.00;bus to college', 'transport', 16.0),
    ('2026-09-22;food;52.00;lunch at canteen', 'food', 52.0),
    ('2026-09-22;coffee;45.00;coffee before lecture', 'coffee', 45.0),
]


# Завдання 2
def test_parse_line_valid():
    record = parse_line("2026-09-21;food;85.50;lunch")
    assert record["category"] == "food"
    assert record["amount"] == 85.5


def test_parse_line_all_fields():
    record = parse_line("2026-09-21;food;85.50;lunch")
    assert record == {"date": "2026-09-21", "category": "food", "amount": 85.5, "comment":
"lunch"}


def test_parse_line_amount_is_float():
    assert isinstance(parse_line("2026-09-22;transport;30;bus")["amount"], float)


# Завдання 3
def test_parse_line_bad_amount():
    with pytest.raises(ValueError, match="not a number"):
        parse_line("2026-09-21;food;abc;lunch")
```

```
def test_parse_line_too_few_fields():
    with pytest.raises(ValueError, match="expected 4 fields"):
        parse_line("2026-09-21;food")

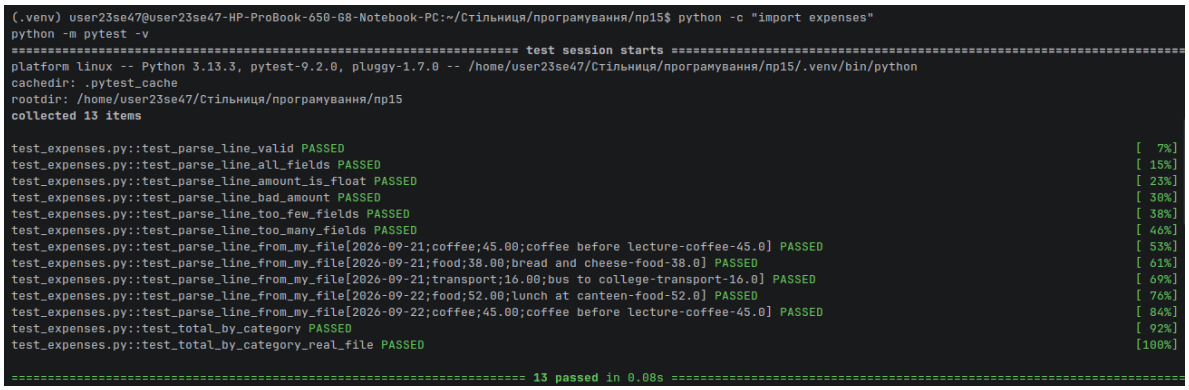

def test_parse_line_too_many_fields():
    with pytest.raises(ValueError, match="expected 4 fields"):
        parse_line("2026-09-21;food;85.50;lunch;extra")


# Завдання 4
@pytest.mark.parametrize(("line", "category", "amount"), CASES)
def test_parse_line_from_my_file(line, category, amount):
    record = parse_line(line)
    assert record["category"] == category
    assert record["amount"] == pytest.approx(amount)

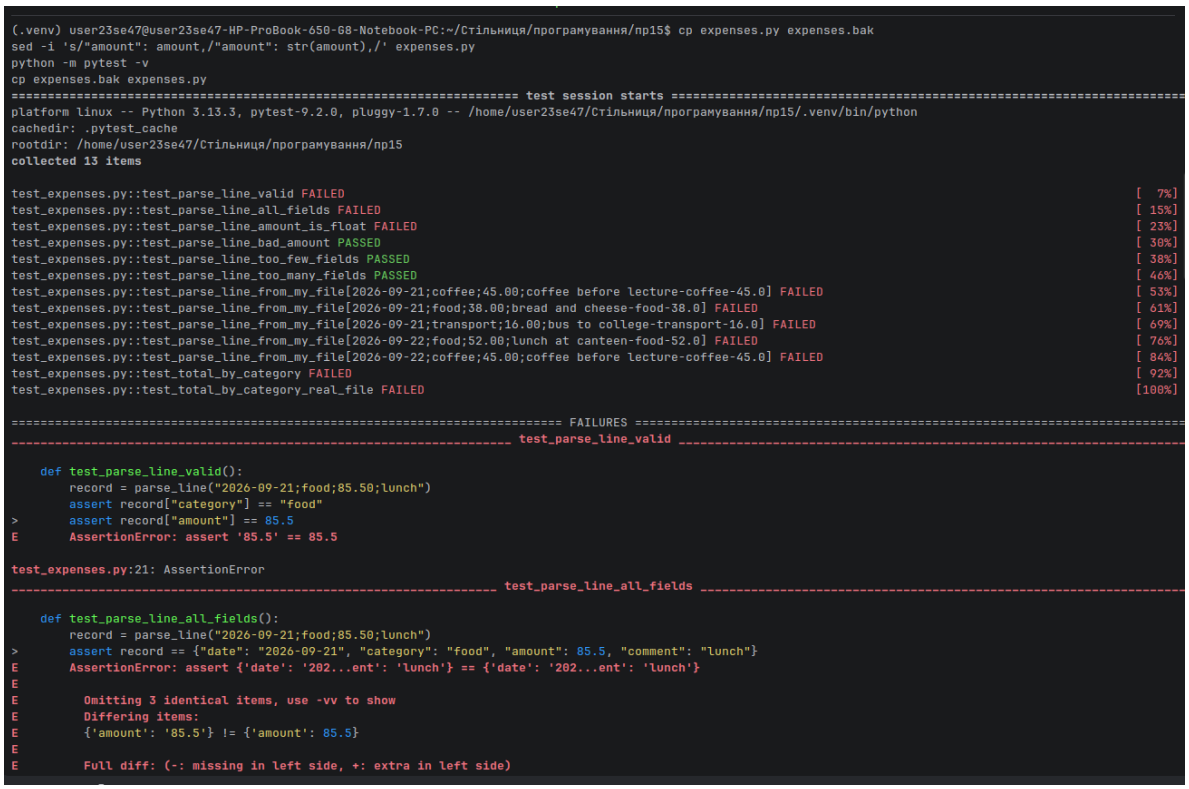

# Завдання 5
def test_total_by_category(tmp_path):
    path = tmp_path / "expenses.csv"
    path.write_text(
        "# date;category;amount;comment\n"
        "2026-09-21;food;85.50;lunch\n"
        "\n"
        "2026-09-22;food;14.50;coffee\n",
        encoding="utf-8",
    )
    totals = total_by_category(load_expenses(path))
    assert totals == {"food": 100.0}


def test_total_by_category_real_file():
    path = Path(__file__).resolve().parent / "data" / "expenses.csv"
    totals = total_by_category(load_expenses(path))
    assert totals == {
        "coffee": 180.00,
        "food": 198.00,
        "transport": 48.00,
        "snacks": 49.00,
        "stationery": 35.00,
    }
```

Усі тести пройдено:



Тести після навмисного пошкодження функції (початок виводу та підсумок):



```
(.venv) user23se47@user23se47-HP-ProBook-650-G8-Notebook-PC:~/Стільниця/програмування/пр15$ cp expenses.py expenses.bak
sed -i 's/"amount": amount,/"amount": str(amount),/' expenses.py
python -m pytest -v
cp expenses.bak expenses.py
============================= test session starts ==============================
platform linux -- Python 3.13.3, pytest-9.2.0, pluggy-1.7.0 -- /home/user23se47/Стільниця/програмування/пр15/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/user23se47/Стільниця/програмування/пр15
collected 13 items

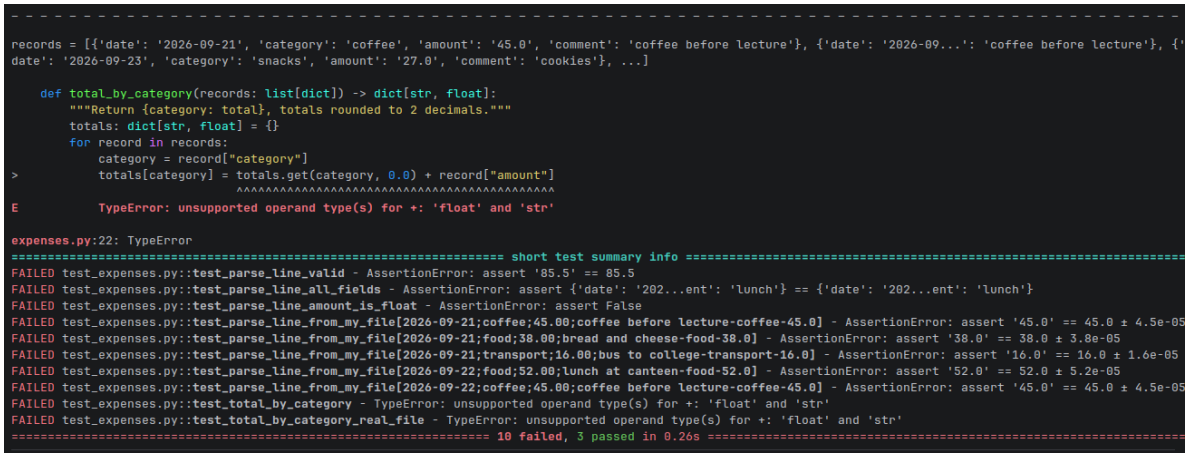
test_expenses.py::test_parse_line_valid FAILED                           [  7%]
test_expenses.py::test_parse_line_all_fields FAILED                      [ 15%]
test_expenses.py::test_parse_line_amount_is_float FAILED                 [ 23%]
test_expenses.py::test_parse_line_bad_amount PASSED                      [ 30%]
test_expenses.py::test_parse_line_too_few_fields PASSED                  [ 38%]
test_expenses.py::test_parse_line_too_many_fields PASSED                 [ 46%]
test_expenses.py::test_parse_line_from_my_file[2026-09-21;coffee;45.00;coffee before lecture-coffee-45.0] FAILED [ 53%]
test_expenses.py::test_parse_line_from_my_file[2026-09-21;food;38.00;bread and cheese-food-38.0] FAILED [ 61%]
test_expenses.py::test_parse_line_from_my_file[2026-09-21;transport;16.00;bus to college-transport-16.0] FAILED [ 69%]
test_expenses.py::test_parse_line_from_my_file[2026-09-22;food;52.00;lunch at canteen-food-52.0] FAILED [ 76%]
test_expenses.py::test_parse_line_from_my_file[2026-09-22;coffee;45.00;coffee before lecture-coffee-45.0] FAILED [ 84%]
test_expenses.py::test_total_by_category FAILED                          [ 92%]
test_expenses.py::test_total_by_category_real_file FAILED                [100%]

================================== FAILURES ===================================
_____________________________ test_parse_line_valid _____________________________

    def test_parse_line_valid():
        record = parse_line("2026-09-21;food;85.50;lunch")
        assert record["category"] == "food"
>       assert record["amount"] == 85.5
E       AssertionError: assert '85.5' == 85.5

test_expenses.py:21: AssertionError
___________________________ test_parse_line_all_fields ___________________________

    def test_parse_line_all_fields():
        record = parse_line("2026-09-21;food;85.50;lunch")
>       assert record == {"date": "2026-09-21", "category": "food", "amount": 85.5, "comment": "lunch"}
E       AssertionError: assert {'date': '202...ent': 'lunch'} == {'date': '202...ent': 'lunch'}
E
E         Omitting 3 identical items, use -vv to show
E         Differing items:
E         {'amount': '85.5'} != {'amount': 85.5}
E
E         Full diff: (-: missing in left side, +: extra in left side)
```



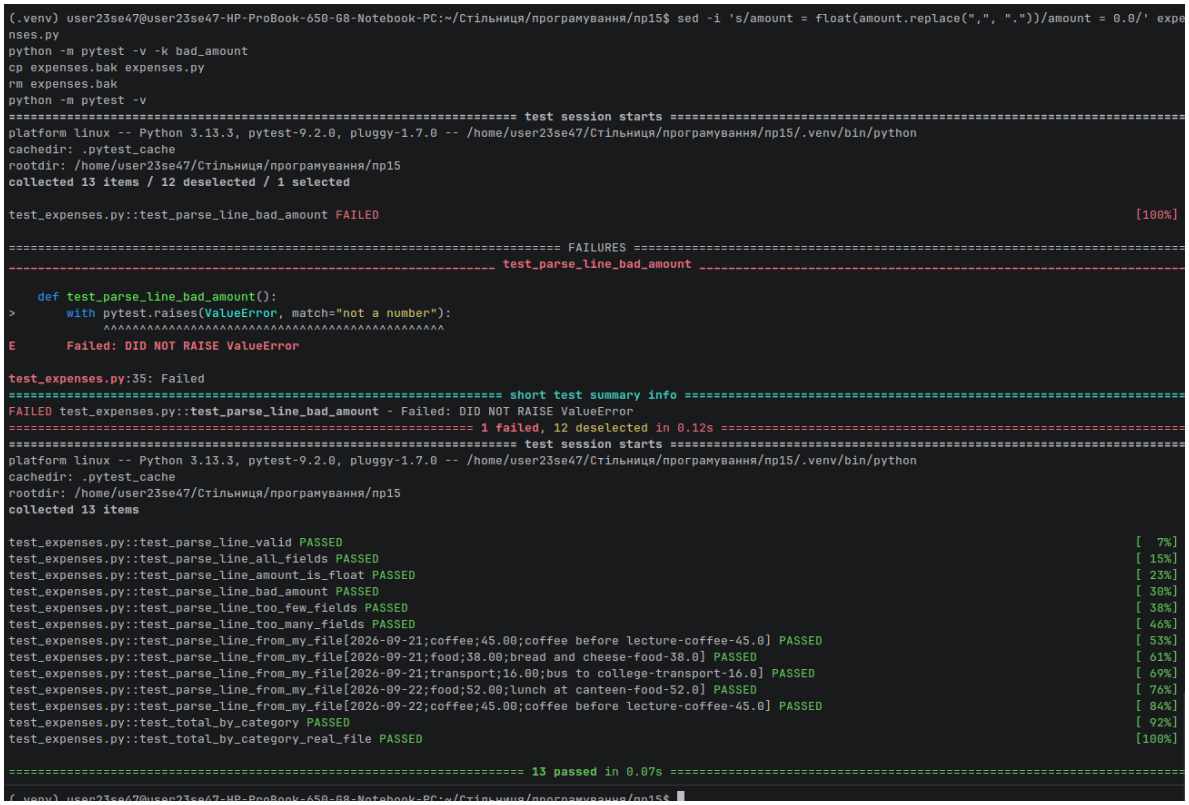
```
records = [{'date': '2026-09-21', 'category': 'coffee', 'amount': '45.0', 'comment': 'coffee before lecture'}, {'date': '2026-09...': 'coffee before lecture'}, {'
date': '2026-09-23', 'category': 'snacks', 'amount': '27.0', 'comment': 'cookies'}, ...]

    def total_by_category(records: list[dict]) -> dict[str, float]:
        """Return {category: total}, totals rounded to 2 decimals."""
        totals: dict[str, float] = {}
        for record in records:
            category = record["category"]
>           totals[category] = totals.get(category, 0.0) + record["amount"]
                               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E           TypeError: unsupported operand type(s) for +: 'float' and 'str'

expenses.py:22: TypeError
=========================== short test summary info ===========================
FAILED test_expenses.py::test_parse_line_valid - AssertionError: assert '85.5' == 85.5
FAILED test_expenses.py::test_parse_line_all_fields - AssertionError: assert {'date': '202...ent': 'lunch'} == {'date': '202...ent': 'lunch'}
FAILED test_expenses.py::test_parse_line_amount_is_float - AssertionError: assert False
FAILED test_expenses.py::test_parse_line_from_my_file[2026-09-21;coffee;45.00;coffee before lecture-coffee-45.0] - AssertionError: assert '45.0' == 45.0 ± 4.5e-05
FAILED test_expenses.py::test_parse_line_from_my_file[2026-09-21;food;38.00;bread and cheese-food-38.0] - AssertionError: assert '38.0' == 38.0 ± 3.8e-05
FAILED test_expenses.py::test_parse_line_from_my_file[2026-09-21;transport;16.00;bus to college-transport-16.0] - AssertionError: assert '16.0' == 16.0 ± 1.6e-05
FAILED test_expenses.py::test_parse_line_from_my_file[2026-09-22;food;52.00;lunch at canteen-food-52.0] - AssertionError: assert '52.0' == 52.0 ± 5.2e-05
FAILED test_expenses.py::test_parse_line_from_my_file[2026-09-22;coffee;45.00;coffee before lecture-coffee-45.0] - AssertionError: assert '45.0' == 45.0 ± 4.5e-05
FAILED test_expenses.py::test_total_by_category - TypeError: unsupported operand type(s) for +: 'float' and 'str'
FAILED test_expenses.py::test_total_by_category_real_file - TypeError: unsupported operand type(s) for +: 'float' and 'str'
========================= 10 failed, 3 passed in 0.26s =========================
```

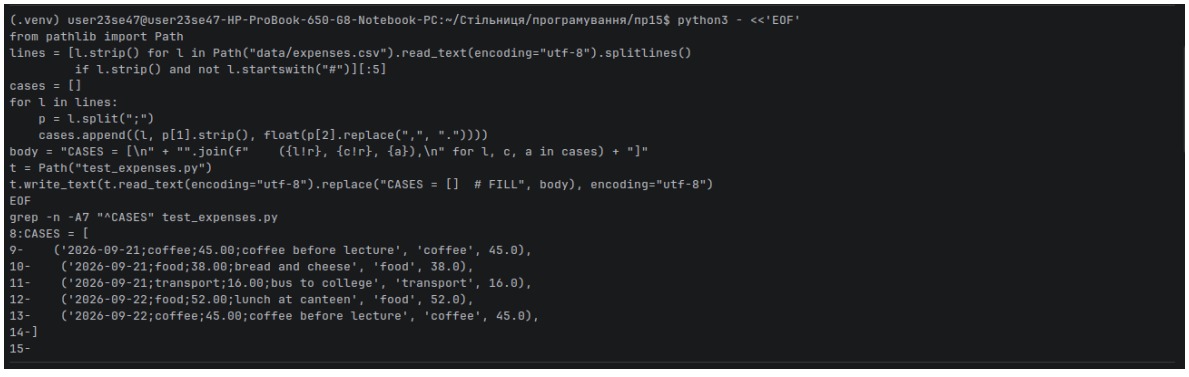
Завдання 3. Перевірка винятків

Я додала тести з pytest.raises для двох видів помилок: нечислова сума (test_parse_line_bad_amount) і неправильна кількість полів, коли їх замало та коли забагато (test_parse_line_too_few_fields і test_parse_line_too_many_fields). Параметр match додатково перевіряє текст повідомлення. Щоб побачити DID NOT RAISE, я прибрала з функції перетворення суми у число (amount = 0.0): тест test_parse_line_bad_amount упав з повідомленням Failed: DID NOT RAISE ValueError. Після відновлення функції всі 13 тестів знову пройшли.



Завдання 4. Параметризований тест

За допомогою @pytest.mark.parametrize я перевірила parse_line на п'яти рядках з мого файлу витрат: для кожного рядка вказано очікувану категорію та суму. Список CASES я згенерувала невеликою програмою, яка бере перші п'ять рядків з data/expenses.csv. pytest -v показав п'ять окремих тестів з різними параметрами в назві (див. скрін із завдання 2).



Завдання 5. Тест з тимчасовим файлом

Тест test_total_by_category створює у тимчасовому каталозі (фікстура tmp_path) файл з коментарем, порожнім рядком і двома витратами на food та перевіряє, що

total_by_category(load_expenses(path)) дає {"food": 100.0}. Тест test_total_by_category_real_file перевіряє функцію на моєму справжньому data/expenses.csv: coffee 180.00, food 198.00, transport 48.00, snacks 49.00, stationery 35.00 (ці суми збігаються з результатом роботи 22 і практичної 14). Обидва тести пройшли (див. скріни вище). Тимчасовий файл після тесту в каталозі пр15 не залишається.

Контрольні запитання

1. Що таке unit-тест і чим він відрізняється від ручної перевірки програми?

Unit-тест (модульний тест) - це код, який викликає одну невелику частину програми, зазвичай одну функцію, з відомими вхідними даними і порівнює результат з очікуваним. Він швидкий, незалежний від інших тестів і повторюваний, а запускається однією командою. Ручна перевірка повільна, потрібно щоразу повторювати всі кроки, легко забути граничний випадок, а результат перевірки ніде не зберігається.

2. Як pytest знаходить тести в каталозі проєкту?

pytest сам шукає тести за іменами від поточного каталогу вглиб: файли test_*.py або *_test.py, у них функції, назва яких починається з test, і класи, назва яких починається з Test (без __init__). Нічого реєструвати не потрібно.

3. Чому тест обов'язково треба побачити таким, що падає?

Тест, який не може впасти, нічого не перевіряє. Якщо навмисно зламати функцію і тест стає червоним, то видно, що він справді помічає помилку. У моїй роботі після заміни числа на рядок 10 тестів впало, а після прибирання перетворення суми тест на виняток упав з DID NOT RAISE.

4. Як перевірити, що функція генерує виняток?

Використати with pytest.raises(ValueError): і викликати функцію всередині блоку. Тест проходить, якщо виникає саме цей виняток; параметр match перевіряє текст повідомлення. Якщо винятку немає, тест падає з повідомленням DID NOT RAISE.

5. Навіщо параметризувати тести замість копіювання однакових функцій?

Декоратор @pytest.mark.parametrize запускає один і той самий тест з різними наборами даних, тому не треба копіювати код. Кожен набір стає окремим тестом зі своєю назвою, тож якщо один впаде, інші все одно виконаються, а у звіті видно, на яких саме даних помилка.

6. Чому тест не повинен залежати від файлів, які можуть змінитися, і як у цьому допомагає tmp_path?

Якщо тест залежить від робочого файлу, який можуть змінити, результат тесту стає нестабільним: сьогодні він проходить, а завтра ні, і тест може пошкодити ваші дані. Фікстура tmp_path дає кожному тесту унікальний тимчасовий каталог: тест сам створює в ньому потрібний файл з відомим вмістом, не чіпає файли проєкту, а після завершення тимчасовий каталог не залишається в проєкті. Єдиний тест на справжньому файлі витрат я залишила тому, що цього вимагає завдання 5.